

Topic 4: Differential Analysis

Lab Course Genome-Oriented Bioinformatics

20.01.2025

This report presents a method to identify differential splicing events from RNA sequencing data by computing Percent Spliced In (PSI) values and applying a binomial-based likelihood ratio statistic (LRS). The approach determines whether reads provide evidence for or against a specific exon skipping event determined from a Gene Transfer Format (GTF) file. The LRS compares two conditions for differential splicing, yielding results that show strong performance relative to existing tools such as DEXSeq. Overall, this method offers a concise workflow for differential splicing analysis and a complementary alternative to widely used software.

Contents

1	Introduction	2
1.1	SAM Format	2
1.2	Read Mapping Types	2
1.3	Evidence of Alternative Splicing	3
2	Materials and Methods	4
2.1	Provided Files	4
2.2	Compute Percent-Spliced-In (PSI) values	5
2.2.1	Program Usage	7
2.3	Simple Differential Test of PSI-Values	7
2.3.1	The Binomial Model for PSI-Values	8
2.3.2	The Reduced Model	9
2.3.3	The Full Model	9
2.3.4	Likelihood Ratio Statistic and Hypothesis Testing	9
2.3.5	Implemented R Functions in <code>LRS.R</code>	9
2.4	Alternative Splicing Comparison	10
2.4.1	DEXSeq Analysis	10
2.4.2	Ground Truth and Matching Results	10
2.4.3	Evaluation Metrics	10
2.4.4	Statistical Analysis and Visualization	11
3	Results	11
3.1	Compute Percent-Spliced-In (PSI) values	11
3.1.1	Runtime	11
3.1.2	Correctness	12
3.2	Simple Differential Test of PSI-Values	12
3.3	Alternative Splicing Comparison	12
3.3.1	ROC Curve Analysis	12
3.3.2	Sensitivity and False Discovery Rate	12

1 Introduction

RNA sequencing (RNAseq) is a powerful and widely used method for analyzing the transcriptome, providing insights into gene expression patterns, alternative splicing, and the discovery of novel transcripts. By converting RNA into complementary DNA (cDNA) and sequencing it using high-throughput platforms, researchers can quantify and characterize the complete set of RNA transcripts (those that are sequenceable) present in a sample. The results of an RNAseq experiment are a set of sequenced reads in the FASTQ format, either single-end or paired-end. These reads are then mapped back onto the reference genome using a mapper such as **STAR** [1].

A typical RNAseq workflow includes sample extraction, sequencing, mapping, annotation, and downstream analyses such as differential expression analysis. In a perfect scenario, each read originates from exactly one transcript. However, sometimes it is unclear from which transcript or even gene a read originates. Additionally, annotations can be incomplete, sequencing or mapping errors can occur, and genes or transcripts can overlap, complicating the mapping process. The annotation steps attempt to quantify and correct these errors, sort out improbable or incorrect reads, and assign reads to genes.

Strand specificity in RNA sequencing refers to the ability to determine the strand of DNA from which an RNA transcript originates. This property is important because transcription is inherently directional, with RNA synthesized from only one DNA template strand. Strand-specific sequencing protocols preserve this directionality, allowing researchers to distinguish between overlapping genes transcribed on opposite strands or sense and antisense transcription. The need for strand specificity arises from the complexity of eukaryotic genomes, where overlapping genes, antisense transcription, and non-coding RNAs are common. It can be challenging to accurately assign reads to their source transcripts without strand-specific information, leading to ambiguous or erroneous interpretations. By maintaining strand specificity, RNAseq enables a more precise understanding of transcriptional regulation, splicing, and gene expression patterns. Importantly, all experiments used in this project are unstranded.

1.1 Sequence Alignment Map Format

The Sequence Alignment Map (SAM) format is a widely used text-based format for storing biological sequences that are aligned to a reference genome. Each entry in a SAM file contains detailed information about a read and its alignment, such as the alignment positions and possible mismatches. While SAM files are human-readable, they are relatively large in size, as each field is stored as plain text.

BAM files are the binary equivalent of SAM files, designed to store the same alignment information in a compressed and efficient manner. Unlike the plain-text SAM format, BAM files use binary encoding to represent data, significantly reducing their storage requirements. For example, when storing numbers in plaintext encoded in UTF-8, the number '64' would be encoded as the characters '6' and '4', each requiring 8 bits. In contrast, the binary representation of 64 would be 01000000, which is 8 bits in total. This is an oversimplified example but it illustrates some of the advantages of the binary format. BAM files can be indexed and sorted by name or position, enabling fast access to specific regions of interest. However, this means that a single forward/reverse read pair might not appear consecutively, as the reads are sorted by their position on the reference genome. Several read inconsistencies occur in BAM files that should be excluded from analysis:

- Reads in a pair mapping to different chromosomes.
- Both reads in a pair mapping to the same strand.
- Overlapping reads in a pair that imply different introns (see Figure 1).

When annotating, the goal is to identify the most plausible explanation for each read based on its alignment and features. A read may originate from a transcript, reflecting active transcription within an annotated gene, or from a gene's transcribed region but not its exons, such as an intronic region or a novel isoform—especially if splicing events are observed. Alternatively, reads may align antisense to known transcripts, potentially indicating by-products or genuine antisense transcription. Some reads may be classified as intergenic, often aligning near gene boundaries. This could signal novel isoforms or unannotated untranslated regions (UTRs).

1.2 Read Mapping Types

To accurately classify read pairs, it is essential to define the different types of mappings. A consistent read pair can be classified as **genic**, **intergenic**, or **intergenic containing**. A genic read pair is one where both reads in a pair are contained within a gene. Formally, this means all mapped bases of the read are greater than or equal to the gene start position and less than or equal to the gene end position (inclusive). A read pair is **intergenic** if it is not **genic** and does not completely contain one or more genes. A read contains a gene if the first mapped base of the read is less than or equal to the start of the gene and the last mapped base is greater than or equal to the end of the gene. All other read pairs are **intergenic-contained**.

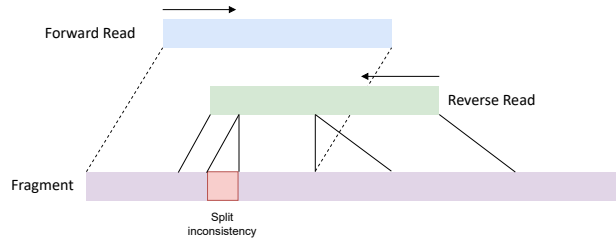


Figure 1: Diagram showing an example of an inconsistent read where the reads in a pair overlap but imply different introns (marked in red).

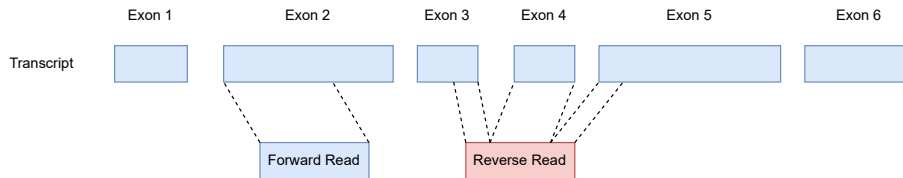


Figure 2: Diagram showing an example of a transcriptomic read pair. The read pair is fully contained within a gene and matches contiguously to a specific transcript section. The color indicates the strand.

A **genic** read pair can be further classified as **transcriptomic**, **merged-transcriptomic**, or **intronic**. A read pair is considered **transcriptomic** if it is fully contained within at least one annotated transcript and aligns contiguously to a specific transcript sequence section (see Figure 2). This means that the intron boundaries observed in the read pair are consistent with those defined in the transcript annotation. Formally, a read pair matches a transcript if both read Region Vectors (RVs) are sub-RVs (defined later) of at least one transcript.

A read pair is classified as **merged-transcriptomic** if it aligns to the merged transcript of a gene but does not necessarily match contiguously (Figure 3). The merged transcript represents the union of all exonic regions of a gene, disregarding specific intron boundaries. This classification accounts for reads that may span novel isoforms or unannotated transcripts. Formally, a read pair matches the merged transcript if each base of the read pair aligns with some exon in the merged transcript. This broader definition enables the identification of potential new transcript variants while maintaining consistency with the gene's exon structure.

A read pair is classified as **intronic** if it is fully contained within the gene body but does not align to any annotated transcript or merged transcript (Figure 4). Intronic reads may represent unprocessed RNA, novel isoforms with unannotated intron retention, or other transcriptional events within the gene body.

Since we are only looking for the most biologically plausible explanation, we only want to consider the most plausible case (**transcriptomic** > **merged-transcriptomic** > **intronic**). For example, if a read pair matches a specific transcript of one gene but also the merged transcript of another gene, we would only consider the transcriptomic match as it is more plausible. If a read pair matches multiple transcripts of multiple genes, we would want to know all the matched transcripts.

1.3 Evidence of Alternative Splicing

Since reads can map to specific transcripts of a gene, it is possible that a read might indicate the inclusion or exclusion of a specific exon. An example of an evidence for an exon skipping event is shown in Figure 5. Some reads provide evidence for the inclusion of an exon (inclusion reads), while others provide evidence for the exclusion of an exon (exclusion reads). Additionally, even though some reads might map to a gene, they do not provide evidence for the inclusion or exclusion of a specific exon. These reads are considered ambiguous.

The Percent Spliced In (PSI) value is a metric used to quantify the inclusion of an exon in a transcript. It

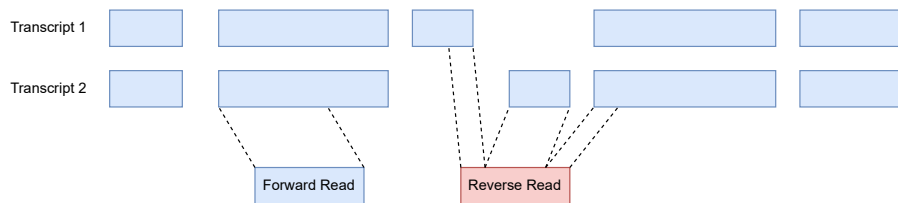


Figure 3: Diagram showing an example of a merged transcriptomic read pair. The read pair aligns to the merged transcript of a gene but does not necessarily match contiguously. The color indicates the strand.

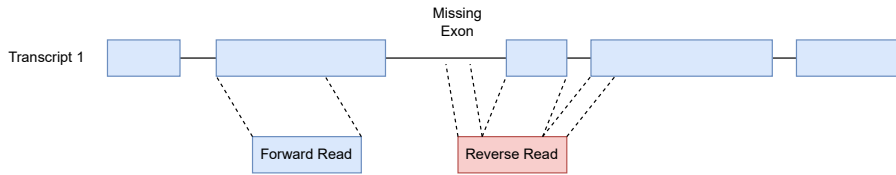


Figure 4: Diagram showing an example of an intronic read pair. The read pair is fully contained within the gene body but does not align with any annotated transcript or merged transcript. The color indicates the strand.

is calculated using the formula shown in Figure 6. The PSI value ranges from 0 to 1, where 0 indicates that the exon is always excluded, and 1 indicates that the exon is always included. The PSI value is calculated by dividing the number of inclusion reads by the sum of inclusion and exclusion reads. The PSI value is calculated for each exon in an exon skipping event.

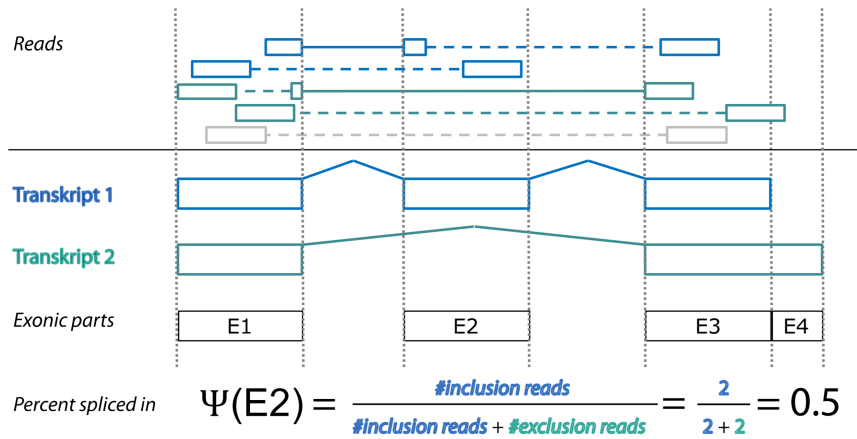


Figure 5: Diagram showing an example exon skipping event and which reads are considered as included, excluded, or ambiguous for the PSI calculation. Transcript 1 would be the wild type, while Transcript 2 would be the splicing variant. Exon 2 is skipped in Transcript 2. The last read is considered ambiguous, as it matches to both Transcript 1 and 2.

$$\Psi(E) = \frac{I}{I + E}$$

(1)

Figure 6: Ψ value for a given exon E is calculated by dividing the number of included reads I by the sum of included and excluded reads $I + E$.

The goal of this project is to analyze a RNAseq experiment, identifying whether a read provides evidence for the inclusion or exclusion of a specific exon in a gene, and compare this approach to previously established methods.

2 Materials and Methods

2.1 Provided Files

The following files were provided for the analysis of alternative splicing events and served as key resources for the various analyses:

- **bam/**: This directory contains BAM files, which represent the aligned sequencing reads.

- **annotation.b37.gtf**: This file contains the transcript annotation in GTF format. It provides structural information about transcripts, including exon and intron boundaries, which are critical for mapping reads to specific genomic features.
- **differential.exons.RData**: This RData file includes the ground truth for the exons identified in the annotation file. It serves as a reference for validating the results of exon-level differential expression analyses.
- **diff.psi.test.RData**: This file contains data for testing the method for likelihood ratio testing (LRS) in the context of differential splicing.
- **AlternativeSplicing/**: Contains partial reference solutions for testing.

2.2 Compute Percent-Spliced-In (PSI) values

Figure 8 shows the general process for solving the task. Due to the complexity of this assignment and the numerous cases, a flowchart is optimal for understanding the task. This allows optimal structuring of an implementation, as one can see which checks can be performed first, in order to possibly rule out other checks from needing to be made, saving computation time. The flowchart shows the process of processing a single read but differs from the actual implementation due to structural reasons. This is because the implementation processes the reads in batches (by chromosome), instead of one by one. The general structure of the program is described below:

1. **Initialization and Setup**: Creation of the **AlternativeSplicingAnnotator** object which initializes the **GTFAnnotation**, calculates the exon skipping events (as per the first report), as well as initializing the **SamReader**, and **IntervalTreeForestManager**. The exon skipping events are stored so that each individual exon that is skipped as a part of an exon skipping event has its own inclusion and exclusion count.
2. **Iteration Process**:
 - (a) **Iterate through the SamReader**: Iterate to the next record, unless the end of the file is reached in which case go to step 4
 - (b) **Chromosome Check**: Check if the current record is on the same chromosome as the previous record. If not, process the reads in step 3.
 - (c) **Valid Read Check**: Check conditions if the read is valid. If not, continue to step 2a.
 - (d) **Read Pair Check**: Check if we have seen the record's mate before:
 - If yes, add the records in the correct order (first of pair) to the readsToAnnotate List
 - If no, add the record to the lookup table and continue to the next record.
 - (e) Go to step 2a.
3. **New Chromosome Processing** :
 - (a) **Read Processing**:
 - i. Calculate the Read's attributes such as the combined read intervals, extracting alignment start and end, etc.
 - ii. If there are any **cgenes**, check for a transcriptomic match. Otherwise, skip the the next read.
 - iii. For each matched gene with a transcriptomic match, iterate through the exon skipping events for that gene (by skipped exon).
 - iv. If the matched transcripts equal the Wild Type transcripts of the exon skipping event & the read alignment overlaps the skipped exon, the inclusion count is incremented. Specifically, the start of the read alignment is before or at the end of the skipped exon, and the end of the read alignment is after or at the start of the skipped exon. This ensures that the read covers at least part of the skipped exon.
 - v. If the matched transcripts equal the Splice Variant transcripts of the exon skipping event & the read alignment fully contains the skipped exon, the exclusion count is incremented. Specifically, the start of the read alignment is before the start of the skipped exon, and the end of the read alignment is after the end of the skipped exon. This ensures that the read fully contains the skipped exon and is not an ambiguous read.
 - vi. Repeat this process for all exons in each exon skipping event for each matched gene.
 - (b) Reset the data structures and return to step 2b.
4. **Last Chromosome Processing** :
 - (a) Process the reads like in Step 3a.

- (b) Calculate the PSI score for each exon for each exon skipping event.
- (c) If either the inclusion or exclusion count is greater than 0, the result is printed to the output file in the format shown in Table 1.

Since the iteration process through the BAM file and caching (by chromosome) process is the same as the BamFeatures task, an abstract class, `ReadAnnotator`, was created. The abstract class implements the iterating, read validation checking (see previous report) and stores the required data structures (lookup table, forest manager). While iterating through the BAM file, the abstract class calls up the `processNewChromosome` and `processLastChromosome` methods, which are implemented in the respective annotator (`AlternativeSplicingAnnotator` & `FeatureCountAnnotator`). The exon skipping events are processed in an identical fashion to the first report, with the only difference being that the inclusion and exclusion counts are stored for each exon in the exon skipping event. These events are then stored in a map, where the key is the gene id.

In this task, we consider strand unspecific experiments, meaning that we always check both strands for a gene match. When iterating through the `SAMRecords`, if the read is from a new chromosome, all reads from the previous chromosome are processed. Since the file is organized by chromosome and position, the lookup data structures can be reset. Before adding a read to the lookup table, it is checked for validity:

1. The read is mapped
2. The read is part of a pair
3. The mate read is mapped
4. The reads are on different strands
5. The reads are on the same chromosome
6. The read is not a supplementary alignment

This check ensures that only valid reads are processed, guaranteeing the correctness of the annotation. Since both records of the first and second read in a pair are needed, processing cannot begin until both reads are found. A lookup table is used to match reads with their mates. If the mate has not been found yet, the read is added to the table using the read name, which is the same for paired reads but unique across the experiment. Once the mate is found, both reads are added to the `readsToAnnotate` list in the correct order. After processing the entire chromosome, all pairs in `readsToAnnotate` are annotated, and any remaining entries in the lookup table are ignored.

The annotation of a read pair begins with extracting the alignment's start and end positions. These coordinates are then used to find the genes on the current chromosome that fully include the read pair in their intervals. If there are any containing genes, we check if there is a transcriptomic match. To classify a read as transcriptomic, both reads' region vectors must be sub-region vectors of at least one transcript. A read region vector r is considered a sub-region vector of a transcript t if $\text{cut}(t, r_{\text{start}}, r_{\text{end}}) = r$, as shown in Figure 7. For all

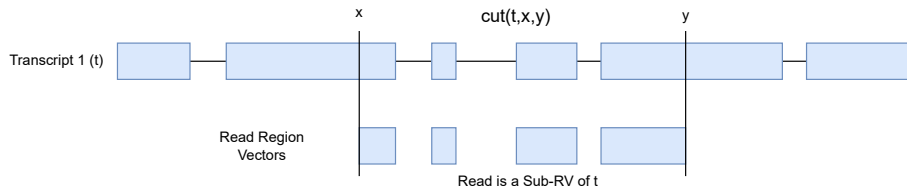


Figure 7: Diagram showing how the cut method works for checking if a read matches a transcript. The transcript is cut starting from the first base of the read and ending at the last base of the read. If the cut transcript is the same as the read, the read matches the transcript.

the matching genes (in case there are multiple), we iterate through the exon skipping events. If the matching transcripts (from the transcriptomic match) equal the Wild Type transcripts of the exon skipping event and the read alignment overlaps the skipped exon, the inclusion count is incremented. If the matching transcripts equal the Splice Variant transcripts of the exon skipping event and the read alignment fully contains the skipped exon, the exclusion count is incremented. This process is repeated for all exons in each exon skipping event for each matched gene. After processing all reads, the PSI score is calculated for each exon in each exon skipping event. If either the inclusion or exclusion count is greater than 0, the result is printed to the output file in the format shown in Table 1.

Table 1: Output format for the .psi files (tab separated)

gene	exon	num_incl_reads	num_excl_reads	num_total_reads	psi
------	------	----------------	----------------	-----------------	-----

2.2.1 Program Usage

usage: java -jar differentialanalysis.jar [-h] -gtf <gtf_file> -bam <bamfile> -o <output_tsv>

named arguments:

-h, --help	show this help message and exit
-gtf <gtf_file>	GTF file
-bam <bamfile>	BAM file
-o <output_tsv>	Output file

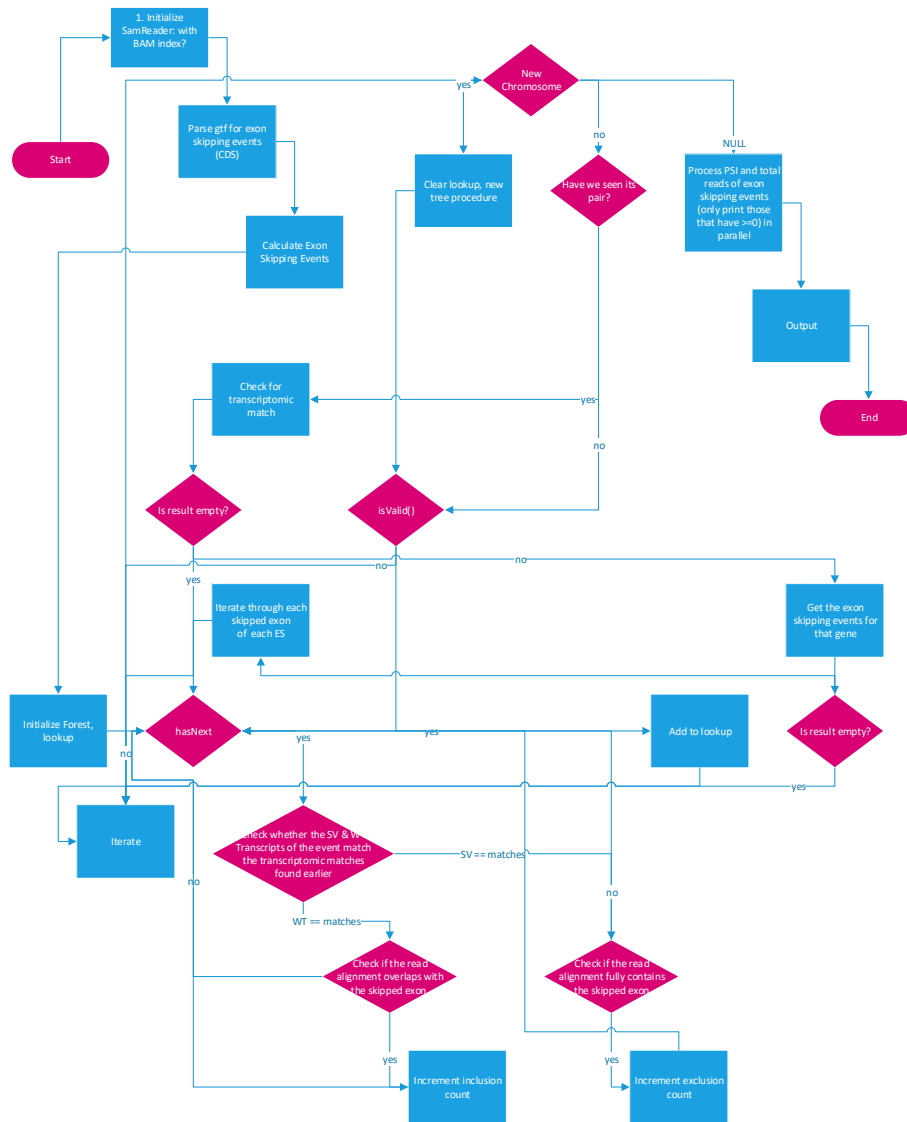


Figure 8: Flowchart illustrating the logical program flow for solving the Alternative Splicing Assignment. Rectangles represent procedures, diamonds represent decisions, and rounded blocks represent start and endpoints. The process involves parsing the GTF, calculating the exon skipping events and iterating through the BAMFile. Once both forward and reverse reads have been found, the processing is performed.

2.3 Simple Differential Test of PSI-Values

To evaluate whether the Percent Spliced In (PSI) values differ between two groups, we assume a simple statistical model based on the binomial distribution. This test compares two probability models: a reduced model and a full model.

2.3.1 The Binomial Model for PSI-Values

The probability of observing i inclusion reads out of N total reads, given the inclusion probability p , is described by the binomial distribution:

$$P(i|p, N) = \binom{N}{i} \cdot p^i \cdot (1-p)^{N-i} \quad (2)$$

where $\binom{N}{i}$ is the binomial coefficient, p is the inclusion probability, and N is the total number of reads (inclusion + exclusion reads). The inclusion probability p corresponds to the PSI ratio. This formula provides the foundation for both models. With the assumption of independence between reads, the likelihood of the observed data is calculated as the product of the binomial probabilities for all samples. To find the MLE of the inclusion probability p for the binomial distribution, we maximize the likelihood function:

$$L(p) = \prod_{j=1}^n \binom{N_j}{i_j} p^{i_j} (1-p)^{N_j-i_j}, \quad (3)$$

where n is the number of samples, i_j is the inclusion count, and N_j is the total read count for sample j .

Taking the natural logarithm of the likelihood simplifies the product to a sum:

$$\log L(p) = \sum_{j=1}^n \left[\log \binom{N_j}{i_j} + i_j \log(p) + (N_j - i_j) \log(1-p) \right]. \quad (4)$$

The binomial coefficient $\binom{N_j}{i_j}$ does not depend on p and can be ignored during differentiation. The relevant terms are:

$$\log L(p) = \sum_{j=1}^n [i_j \log(p) + (N_j - i_j) \log(1-p)]. \quad (5)$$

To maximize $\log L(p)$, take the derivative with respect to p and set it to zero:

$$\frac{\partial \log L(p)}{\partial p} = \sum_{j=1}^n \left[\frac{i_j}{p} - \frac{N_j - i_j}{1-p} \right] = 0. \quad (6)$$

Simplifying and solving for p :

$$\sum_{j=1}^n \frac{i_j}{p} = \sum_{j=1}^n \frac{N_j - i_j}{1-p}, \quad (7)$$

$$\frac{\sum_{j=1}^n i_j}{p} = \frac{\sum_{j=1}^n (N_j - i_j)}{1-p}, \quad (8)$$

$$\frac{\sum_{j=1}^n i_j}{p} = \frac{\sum_{j=1}^n N_j - \sum_{j=1}^n i_j}{1-p}. \quad (9)$$

Let $I = \sum_{j=1}^n i_j$ (total inclusion reads) and $N = \sum_{j=1}^n N_j$ (total reads). Substitute these terms:

$$\frac{I}{p} = \frac{N - I}{1-p}. \quad (10)$$

Cross-multiply and simplify:

$$I(1-p) = (N-I)p, \quad (11)$$

$$I - Ip = Np - Ip, \quad (12)$$

$$I = Np, \quad (13)$$

$$p = \frac{I}{N}. \quad (14)$$

Thus, the MLE for p is:

$$\hat{p} = \frac{\sum_{j=1}^n i_j}{\sum_{j=1}^n N_j}. \quad (15)$$

2.3.2 The Reduced Model

The reduced model assumes that all samples share a common inclusion probability p_0 . Under this model, the likelihood of the observed data for all samples is:

$$L_{reduced}(p_0) = \prod_{j=1}^{10} P(i_j, N_j | p_0) \quad (16)$$

where i_j and N_j are the inclusion and total reads for sample j . Using the log-likelihood simplifies computation:

$$\log L_{reduced}(p_0) = \sum_{j=1}^{10} \left[\log \binom{N_j}{i_j} + i_j \log(p_0) + (N_j - i_j) \log(1 - p_0) \right] \quad (17)$$

The maximum likelihood estimate (MLE) for p_0 is calculated by maximizing this log-likelihood:

$$p_0 = \frac{\sum_{j=1}^{10} i_j}{\sum_{j=1}^{10} N_j} \quad (18)$$

2.3.3 The Full Model

The full model allows for separate inclusion probabilities p_1 and p_2 for the two groups. As an example, consider two groups of five samples each:

$$\text{Group 1: } \{p_1, p_1, p_1, p_1, p_1\}, \quad \text{Group 2: } \{p_2, p_2, p_2, p_2, p_2\}$$

The likelihood of the data under the full model is:

$$L_{full}(p_1, p_2) = \prod_{j=1}^5 P(i_j, N_j | p_1) \cdot \prod_{j=6}^{10} P(i_j, N_j | p_2) \quad (19)$$

The corresponding log-likelihood is:

$$\log L_{full}(p_1, p_2) = \sum_{j=1}^5 \left[\log \binom{N_j}{i_j} + i_j \log(p_1) + (N_j - i_j) \log(1 - p_1) \right] + \sum_{j=6}^{10} \left[\log \binom{N_j}{i_j} + i_j \log(p_2) + (N_j - i_j) \log(1 - p_2) \right] \quad (20)$$

The MLEs for p_1 and p_2 are:

$$p_1 = \frac{\sum_{j=1}^5 i_j}{\sum_{j=1}^5 N_j} \quad (21)$$

$$p_2 = \frac{\sum_{j=6}^{10} i_j}{\sum_{j=6}^{10} N_j} \quad (22)$$

2.3.4 Likelihood Ratio Statistic and Hypothesis Testing

To test whether the PSI values differ between the two groups, we compute the likelihood ratio statistic (LRS):

$$LRS = -2 \cdot \log \left(\frac{L_{reduced}}{L_{full}} \right) \quad (23)$$

which simplifies to:

$$LRS = -2 \cdot (\log L_{reduced} - \log L_{full}) \quad (24)$$

Under the null hypothesis ($H_0 : p_1 = p_2$), the LRS follows a Chi-square distribution with one degree of freedom ($df = 1$). The p-value is calculated as:

$$P(LRS > x) = 1 - F_{\chi^2}(x, df = 1) \quad (25)$$

where F_{χ^2} is the cumulative distribution function of the Chi-square distribution.

2.3.5 Implemented R Functions in LRS.R

The analysis pipeline is built using two main functions:

LRS The **LRS** function performs the likelihood ratio test for a single exon. It takes three inputs:

- **incl**: A vector of inclusion read counts (IRC) for each sample.
- **total**: A vector of total read counts (IRC + ERC) for each sample.
- **group**: A vector indicating the group assignment (1 or 2) of each sample.

Using these inputs, the function:

1. Computes p_0 , p_1 , and p_2 as maximum likelihood estimates.
2. Calculates the log-likelihoods for the reduced and full models.
3. Derives the likelihood ratio statistic (LRS) and the corresponding p-value using the Chi-square distribution.

The function returns a list containing p_0 , p_1 , p_2 , $\log L_{reduced}$, $\log L_{full}$, LRS, and the p-value.

diff.splicing The **diff.splicing** function extends the **LRS** method for a dataset of multiple PSI files generated using the program described in Section 2.2. It:

1. Aggregates data from all input files (**psi.files**), merging metrics for each exon across samples.
2. Processes each exon using **LRS**, storing results in a structured table.
3. Adjusts p-values for multiple testing (Benjamini-Hochberg) and returns a final result with all columns listed below.

The final output is a table containing:

- **gene, exon**: Identifiers for each exon being analyzed.
- p_0, p_1, p_2 : Maximum likelihood estimates of the inclusion probabilities under the reduced and full models.
- $\log L_{reduced}, \log L_{full}$: Log-likelihoods for the reduced and full models.
- **lrs**: Likelihood ratio statistic, quantifying the improvement of the full model over the reduced model.
- **pvalue**: Statistical significance of the observed LRS.
- **padj**: Adjusted p-values for controlling the false discovery rate (FDR).

2.4 Comparison With Other Alternative Splicing Tools

To evaluate the performance of our likelihood ratio statistic (LRS) method, we compared it with the DEXSeq tool using simulated BAM files provided in the assignment. The analysis was conducted as follows:

2.4.1 DEXSeq Analysis

The DEXSeq[2] tool was used to perform differential splicing analysis on the same BAM files processed in the LRS pipeline. Adjusted p-values (**padj**) were obtained for each exon. Genomic ranges were extracted to align DEXSeq results with the exons analyzed by our LRS method.

2.4.2 Ground Truth and Matching Results

Ground truth labels indicating truly differentially skipped exons were provided in **differential_exons.RData**. To compare the two methods:

- LRS and DEXSeq results were merged using gene identifiers and exon ranges.
- A ground truth flag was added to indicate whether an exon was part of the true differentially skipped set.

2.4.3 Evaluation Metrics

We computed the following metrics:

- **Sensitivity**: The proportion of true positives (correctly identified differentially skipped exons) among all actual positives.
- **False Discovery Rate (FDR)**: The proportion of false positives among all predicted positives.
- **ROC Curves and AUC**: ROC curves were generated to visualize the trade-off between true positive rate (TPR) and false positive rate (FPR). The area under the curve (AUC) was calculated to quantify overall performance.

Table 2: Runtime for the annotation on the provided samples 1-10. with the number of records The BAM file were processed on the bioclient grid (write/read to/from HDD).

Sample Number	1	2	3	4	5	6	7	8	9	10
Annotation Time (ms)	647	644	685	462	465	513	520	537	519	735
Total Records	188,242	220,274	190,272	188,538	189,678	187,512	220,924	220,800	221,470	219,662

2.4.4 Statistical Analysis and Visualization

The R `ROCR`[3] library was used to compute performance metrics and generate ROC curves. The comparison process involved:

1. Generating ROC curves for both methods using `padj` values as classifiers.
2. Plotting the curves on the same graph for visual comparison, with AUC values included in the legend.
3. Calculating sensitivity and FDR for predictions with `padj` < 0.05.

3 Results

3.1 Compute Percent-Spliced-In (PSI) values

3.1.1 Runtime

Table 2 reports the runtime for annotating samples 1-10. The annotation time for each sample is relatively consistent, with an average runtime of 572.7 ms. Additionally, the total number of records processed for each sample is provided. It is important to note that the reported runtime excludes the time required to parse the GTF file. Since the same GTF file is used across all samples, the parsing time is constant and independent of the experiment size (BAM file). The slight differences in runtime do not always correlate with the experiments having the most records. This discrepancy might be due to some experiments containing more invalid records, which are easier to process. However, the minor variations in runtime can most likely be attributed to randomness and differences in the servers on the grid that executed the program.

The annotation process is primarily influenced by the size of the BAM file, as larger files contain more reads (records) to process. For each read in the BAM file, the annotation pipeline performs several operations:

1. **Validation of Read:** Ensures that the read meets format requirements. This step has a constant time complexity, $O(1)$, per read.
2. **Tree Search for Matches:** Checks the read against the interval tree to identify matching transcript regions. This step has a time complexity of $(O(\log(M)))$, where M is the amount of genes on the chromosome.
3. **Transcriptomic Match Check:** Verifies whether the matched regions are transcriptomic, which involves comparing the alignment blocks of the read against annotated transcripts. This step scales with the number of alignment blocks per read (B). If B is relatively small and consistent across reads, this step is effectively $O(1)$.
4. **Exon Skipping Event Inclusion Check:** For each matched gene, the program determines whether the read supports or contradicts the exon-skipping events of that gene. This is a set comparison operation for each skipping event of the gene. If there are E exon-skipping events for a gene, this step has $O(E)$ complexity per matched gene.

Assuming the average number of genes matched per read is G , the total runtime complexity for processing N reads can be expressed as:

$$O(N \cdot (\log M + B + G \cdot E))$$

where:

- M : Average/Max number genes on a chromosome.
- B : Average/Max number of alignment blocks per read.
- G : Average/Max number of matched genes per read.
- E : Average/Max number of exon-skipping events per gene.

In practice, the values of B , G , and E are relatively small and constant for most datasets, which simplifies the runtime to being **linear with respect to the number of reads (N)** for large N .

The measured average runtime of 572.7 ms reflects the efficiency of the implementation for the dataset used in the study. Since parsing the GTF file is constant and excluded from the runtime, the reported time accurately reflects the time spent annotating reads.

As the BAM file size increases, the runtime will scale proportionally to the number of reads, assuming that M , B , G , and E remain consistent. However, factors such as memory access patterns, cache usage, and the efficiency of the data structures used for the interval tree and set operations can introduce slight nonlinearities in practical scenarios.

3.1.2 Correctness

Wild Type (WT) transcripts are defined as those that include all exons within the exon skipping event, while Splice Variant (SV) transcripts are those that omit one or more of these exons. If the matching transcripts of a read equal WT transcripts of an exon skipping event, it implies that it doesn't match the SV transcripts. If the read were to exclude one of the exons in the exon skipping event, it should match the SV transcripts. If it matches part of WT and SV, we can naturally conclude it is an ambiguous read. To ensure that the read provides evidence of inclusion of a specific exon(s) in an event, we have to check whether the read alignment overlaps with the skipped exon. The process for an exclusion is analogous, but this time it is checked whether the read alignment fully contains the skipped exon. This WT/SV approach provides a computationally efficient way to determine the inclusion/exclusion status of a read while also ensuring correctness.

3.2 Simple Differential Test of PSI-Values

The LRS statistic was calculated for the provided example files (`psi_data.csv`) in the required format. Applying the Benjamini-Hochberg correction to the p-values yielded adjusted p-values (`padj`). Given an α of 0.05, the number of significant exons decreased from 72 to 42.

3.3 Comparison With Other Alternative Splicing Tools

3.3.1 ROC Curve Analysis

The ROC curves for the LRS-based method (*p_lrs*, red) and DEXSeq (*pd_dseq*, blue) are presented in Figure 9. The diagonal gray line represents the performance of a random classifier.

- The LRS method demonstrates a consistently better ability to distinguish true positives from false positives, with an area under the curve (AUC) of 0.866.
- The DEXSeq method achieves an AUC of 0.736, which indicates lower discriminatory power overall.

3.3.2 Sensitivity and False Discovery Rate

Using an adjusted p-value threshold of `padj` < 0.05, the sensitivity and false discovery rate (FDR) for each method were computed:

- **DEXSeq Method:**
 - Sensitivity: 1.0 (all true positives detected).
 - FDR: 0 (no false positives).
- **LRS Method:**
 - Sensitivity: 0.9677 (96.8% true positives detected).
 - FDR: 0.0323 (3.2% false positives).

The results indicate that both methods effectively identify differentially spliced exons. While the overall AUC was higher for the LRS method, the DEXSeq method performed better given a p-value threshold of 0.05. In the case, the DEXSeq method achieved perfect sensitivity and no false positives, while the LRS method had a slightly lower sensitivity and a small FDR. This comparison establishes the LRS-based approach as a viable alternative to DEXSeq for differential alternative splicing analysis, highlighting the fact that preexisting tools don't always outperform a purpose-built self-developed method.

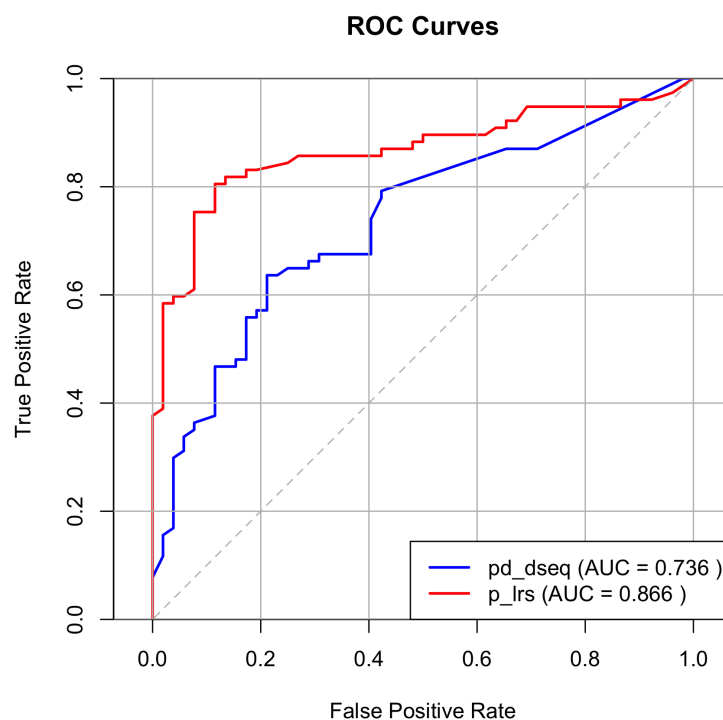


Figure 9: ROC Curves Comparing LRS Method and DEXSeq. The LRS method (red) achieves a higher AUC (0.866) compared to DEXSeq (blue, AUC = 0.736). The diagonal gray line represents a random classifier ($y = x$).

References

- [1] Alexander Dobin et al. "STAR: ultrafast universal RNA-seq aligner". In: *Bioinformatics* 29.1 (Oct. 2012), pp. 15–21. DOI: <https://doi.org/10.1093/bioinformatics/bts635>.
- [2] Michael I Love, Wolfgang Huber, and Simon Anders. "Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2". In: *Genome Biology* 15.12 (Dec. 2014), p. 550.
- [3] Tobias Sing et al. "ROCR: visualizing classifier performance in R". In: *Bioinformatics* 21.20 (Aug. 2005), pp. 3940–3941. ISSN: 1367-4803. DOI: [10.1093/bioinformatics/bti623](https://doi.org/10.1093/bioinformatics/bti623). eprint: https://academic.oup.com/bioinformatics/article-pdf/21/20/3940/48980957/bioinformatics_21_20_3940.pdf. URL: <https://doi.org/10.1093/bioinformatics/bti623>.